

Advanced Process Mining

Assignment Part II

July 5, 2026, 23:59

Prof. Dr. Wil van der Aalst

Chair of Process and Data Science
RWTH Aachen University

Introduction

This assignment part addresses the following topics:

- Stochastic Process Mining: modeling and analysis
- Analyzing object-centric event data
- Declarative Process Mining

As mentioned in the lectures, you may use any software tool of your choice unless a specific tool is required in the task description. For several sub-tasks we recommend writing short scripts to automate the required steps. While doing so, keep the following points in mind:

1. The primary objective is to solve the given task correctly; the solution does not need to be a general-purpose implementation that can be applied to other problems without modification.
2. If a sub-step can be performed more easily by hand, manual execution is perfectly acceptable and may even be preferable.

Assignment Details

- Total number of points obtainable: 100
 - Part I : 60 points
 - Part II: 40 points
- Group size: 2–3
- Deliverables: Document containing your answers, data and models that you created, main code snippets, figures

Tools

In Question 4, you are expected to use the tool **GreatSPN** [1]. An installation guide can be found under <https://github.com/greatspn/SOURCES/blob/master/docs/INSTALL.md>.

Deliverables

- **What to submit**

1. A single PDF document containing all written answers to the assignment questions, hereafter referred to as the *report*.
2. All event log files and process models that you created or used in order to answer the questions.
3. All figures (plots, diagrams, screenshots, etc.) referenced in the report as separate files.
4. The source code (scripts, notebooks, or programs) you wrote for the assignment. If there is no code, you can ignore the following two requirements.

- **Purpose of the code submission**

- The code is provided primarily to demonstrate that the work was performed independently by your group.
- It should be sufficiently commented and structured so that a reviewer can understand the functionality of each component.

- **Execution requirements**

- We do *not* require the code to run from start to finish without additional manual steps.
- It is therefore acceptable if the program produces intermediate artifacts that are later used by another program.
- Nevertheless, the code must be self-contained with respect to the parts that are essential for reproducing the results presented in the PDF.

- **Formatting and organization**

- Submit the deliverables as a compressed archive (e.g., `.zip` or `.tar.gz`) with the following folder structure:

```
assignment_part2_<YourGroupID>/
  report.pdf
  data/
    *.xes, *.csv, *.json ... (event-log files)
  models/
    *.pnml, *.bpmn, *.net ... (process-model files)
  figures/
    *.pdf, *.svg, *.png, ... (all figures)
  code/
    *.py, *.ipynb, ... (source code)
```

Please ensure that the report is readable, that all referenced artifacts are present in the archive, and that submitted code contains enough comments to make its purpose obvious. If you have any questions regarding the required format, feel free to ask questions in the Moodle forum.

Optional Resources

- Jupyter: <https://jupyter.org/index.html>
- GreatSPN: <https://github.com/greatspn/SOURCES>
- PM4Py: <https://github.com/process-intelligence-solutions/pm4py>
- ProM: <https://promtools.org/>

Question:	1	2	3	4	5	Total
Points:	8	5	6	9	12	40

Question 1: Declare (8 points)

- (a) (4 points) Provide two examples of a DECLARE constraint and an event log such that the event log satisfies this constraint, but the process model discovered using the Inductive Miner does not enforce it. That is, the model permits at least one trace that violates the chosen constraint. Furthermore, briefly explain why the model does not enforce the rule.

*To discover a model using the Inductive Miner, use the ProM plugin: **Mine accepting Petri net with Inductive Miner (IM)** with noise threshold 0, author S.J.J. Leemans.*

- (b) (4 points) Now, we consider real-world data. To this end, you have been provided with an event log of a road traffic fine management process: **rtfm.xes.gz**. Discover a Petri net using the Inductive Miner (IM) with noise threshold 0.2 and a DECLARE model and include them in your report. Identify and explain (*approx. 1–2 sentences*) behavior permitted by one model but not the other, and vice versa. Furthermore, include a trace permitted by the DECLARE model, but not the IM model, and one that is permitted by the IM model but not the DECLARE model.

To discover the models, use the following plugins and parameter settings:

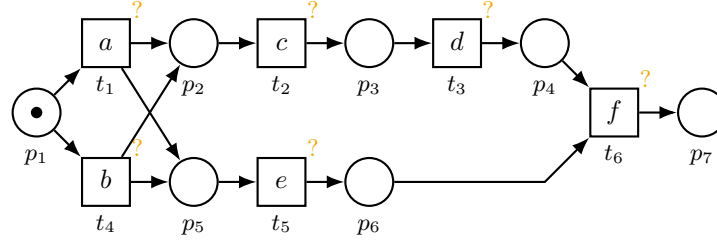
Inductive Miner Use the ProM plugin: **Mine accepting Petri net with Inductive Miner (IM)** with noise threshold 0.2, author S.J.J. Leemans.

Declare Use the ProM plugin: **MINERful Declare Miner** with default settings, author M.H.M. Schouten.

Question 2: EMSC (5 points)

In this exercise, you will investigate how Earth Mover's Stochastic Conformance (EMSC) depends on the weights of a Stochastic labeled Petri Net (SLPN). More specifically, you will determine transition weights that are consistent with a given stochastic language.

Consider the SLPN N_1 shown below:



and the stochastic language L_1 given by

$$L_1: \{a, b, c, d, e, f\}^* \rightarrow [0, 1],$$

$$\begin{aligned} \langle b, c, e, d, f \rangle &\mapsto \frac{6}{35}, & \langle b, c, d, e, f \rangle &\mapsto \frac{3}{35}, \\ \langle a, c, e, d, f \rangle &\mapsto \frac{4}{35}, & \langle b, e, c, d, f \rangle &\mapsto \frac{12}{35}, \\ \langle a, c, d, e, f \rangle &\mapsto \frac{2}{35}, & \langle a, e, c, d, f \rangle &\mapsto \frac{8}{35}. \end{aligned}$$

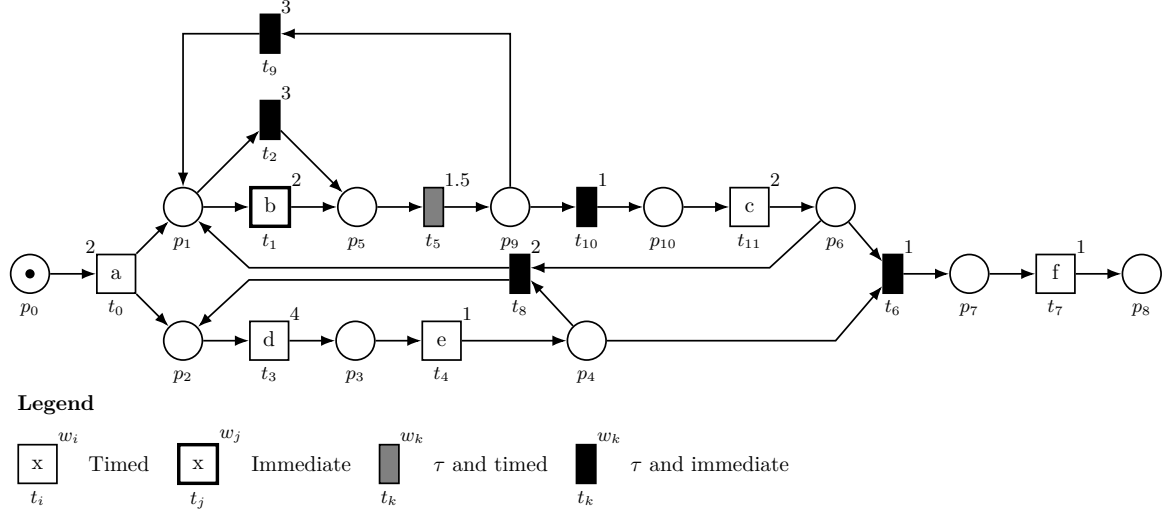
Let M denote the stochastic path language of N_1 , i.e., the set of *all* stochastic paths induced by N_1 , and let δ denote the post-normalized Levenshtein distance as defined in the lecture. Determine transition weights for N_1 that maximize

$$\text{EMSC}_\delta(L_1, M).$$

Question 3: Trace Probabilities in GSLPNs (6 points)

In the lecture, we presented a method for computing the probability of a visible trace in an SLPN with silent paths and loops. This method is based on the embedded discrete-time Markov chain (DTMC) and can be extended directly to Generalized Stochastic Labeled Petri Nets (GSLPNs).

Consider the trace $\sigma = \langle a, d, c, e, f \rangle$ and the GSLPN shown below:



- (a) (3 points) Determine the stochastic path with the highest probability among all paths that project onto the visible trace σ .
- (b) (3 points) Compute the probability of observing the trace σ in this GSLPN.

Note: To the best of our knowledge, there is no dedicated tool available for Part (b). The approach has been presented in the lecture, but you'll need to be a bit creative in how to execute this in practice. On the one hand, one can implement a quite general solution to this problem with much less than 100 lines of code which however requires some manually executed steps. On the other hand, the necessary matrices are still small enough to be computed by hand.

Question 4: Modeling with GSPNs (9 points)

In this question, you will use a Generalized Stochastic Petri Net (GSPN) to model a process, formulate a non-trivial question about it, and answer that question based on the GSPN. Specifically, your task is to answer this question by computing the expected number of tokens in a (singleton) set of places.

Be *creative*: you may choose both the process and the question freely, subject to two fundamental requirements:

- The process must be related in some way to the topic of “*Aachen*”.
- You cannot re-use the “Thesis Process” presented in the lecture. Likewise, minor modifications to this process are not acceptable either.

This requirement should only exclude reusing existing examples. As long as it is satisfied, grading will not depend on whether the “*Aachen*” connection is somewhat artificial.

The process does not need to be realistic (compare the “dog ate my homework” or “thesis process” example from the lecture). The main goal is to practice GSPN modeling and to identify questions that can be answered based on the limiting distribution of GSPN’s continuous-time Markov chain (CTMC). To make the process and question a bit more interesting, your GSPN must satisfy the following additional requirements:

- The GSPN must contain each of the following control-flow features at least once: choice, concurrency, and loop.
 - The GSPN must use at least three of the following four GSPN elements: timed transitions, immediate transitions, inhibitor arcs, and different transition weights.
 - The GSPN must not be safe.
 - The reachability graph (and thus CTMC) of your GSPN must have at least 50 states.
- (a) (5 points) Create a GSPN that satisfies the requirements above using **GreatSPN**. Include an image of the GSPN in your submitted document. In addition, submit the **GreatSPN .net** file.
- (b) (2 points) Briefly describe your model (*approx. 5 sentences*), how it relates to the topic of “*Aachen*” (*approx. 2 sentences*), and state the question you want to answer with it (*approx. 1–3 sentences*).
- (c) (2 points) Answer your question by computing the expected number of tokens in a (singleton) set of places.

Question 5: Object-Centric Process Mining (12 points)

In this question, we investigate a (simplified) process of collaborative coding using a version control system (Git) featuring the object types $OT = \{branches, users, files, file\ versions\}$ and event types $ET = \{commit, pull, merge, push\}$. The behavior is as follows:

“Multiple users work on various branches. These branches reflect work packages or issues. Working on the branches, the users update existing files on these branches. Files are versioned (type file versions). An object-object relation (is_a) indicates for each version object which file it versions. When users commit new file versions, this locally overrides the old file version that the new version is based on. New and old file versions are distinguished at events via the event-object relations produce (new versions) and consume (old versions). After doing one or multiple commits to a branch, users perform a “synchronization with the upstream repository” (activities push, pull and merge). This is done as follows: When users push, all committed files are uploaded to the branch in the repository. This may, however, cause conflicts with other versions of that file on the branch that have been pushed in the meantime. In this case, a merge event is necessary to integrate these changes into a new file version. It is desired that users make these integrations locally: this is done by pulling (and merging, if applicable) directly before a push, instead of merging after the push.”

The process is recorded in the object-centric event log L . For illustration, a fragment of L is depicted below, showing the first events of L together with the corresponding relations. The full event log is provided as an OCEL 2.0 file `version_control.sqlite`.

Fragment of the object-centric event log L .

(a) Events and event-object relations (qualifiers in brackets, if applicable).							(b) Object-object relations.			
Event	Activity	Timestamp	Event-object relations				source	target	qualifier	
			branches	users	files	file versions				
e1	commit	08:34	b1	Tobias	f1	f1_v0 (consume), f1_v1 (produce)		f1_v0	f1	is_a
e2	commit	09:30	b1	Tobias	f1	f1_v1 (consume), f1_v2 (produce)		f1_v1	f1	is_a
e3	pull	09:31	b1	Tobias				f1_v2	f1	is_a
e4	push	09:32	b1	Tobias	f1	f1_v2		f1_v3	f1	is_a
e5	commit	10:20	b1	Cameron	f1	f1_v2 (consume), f1_v3 (produce)		f2_v0	f2	is_a
e6	pull	10:22	b1	Cameron				f2_v1	f2	is_a
e7	push	10:45	b1	Cameron	f1	f1_v3		f2_v2	f2	is_a
e8	commit	11:22	b2	Benedikt	f2	f2_v0 (consume), f2_v1 (produce)		f2_v3	f2	is_a
e9	commit	11:24	b2	Viki	f2	f2_v0 (consume), f2_v2 (produce)				
e10	push	11:30	b2	Benedikt	f2	f2_v1				
e11	push	11:35	b2	Viki	f2	f2_v2				
e12	merge	12:14	b2	Viki	f2	f2_v1 (consume), f2_v2 (consume), f2_v3 (produce)				

The process owner gave the *users* the following constraints for working in the process:

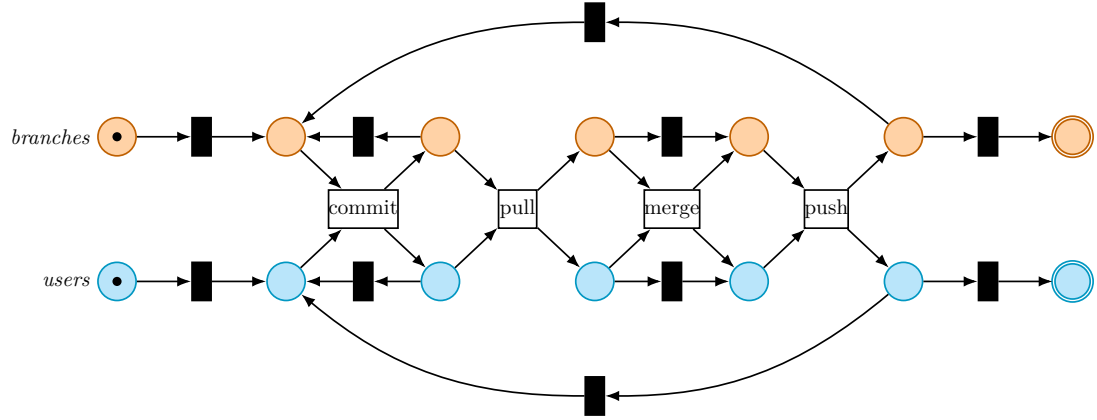
- C1 “You should pull before you push, and you should not do anything between the pull and the push except for possibly a merge.”
- C2 “Do not alternate between branches: After committing to one branch b , you should not touch a different branch until having pushed to b .”
- C3 “Avoid coordination problems: A coordination problem between two users $u_1 \neq u_2$ is a merge event that consumes two versions of the same file, where one version was committed by u_1 and the other version was committed by u_2 .”

- (a) (4.5 points) Check each of the three constraints C1, C2, C3 on L using an OCPQ query. Show the queries with their results and briefly explain each of your solutions (approx. 2–3 sentences per constraint).

Hint: Use the CEL expression editor in OCPQ to express that variables o_1, o_2 should bind to different entities (e.g., $o_1.id() \neq o_2.id()$) and to encode precedence constraints

between events e_1, e_2 ($e_1.time() < e_2.time()$). Use the advanced CEL editor (CEL+) to express more complex expressions over child boxes.¹

- (b) (4.5 points) Consider the following accepting object-centric Petri net (OCPN) N_1 .



Let L_1 denote the log that is obtained by selecting on L the *view*² on the object types *branches* and *users* and all event types. Furthermore, let u be an object of type *users* in L . Let L_1^u be the log that is obtained by first removing from L_1 all events where u is not involved, and then all objects that do not relate to any event, and then all relations that connect to a removed object or removed event.

For $u = \text{Cameron}$, apply the framework for *flattening-based conformance checking* to compute average token-based replay fitness and average alignment-based precision of L_1^u against N_1 . Show all intermediate results per object type (flattened traces, token counts per place, alignments, and prefix automata) and report the aggregated fitness and precision scores. Can you use your results to check the constraints C1 and C2 for user *Cameron* on L ? Justify your answers (*approx. 2–3 sentences per constraint*).

- (c) (3 points) Let L_2 denote the log that is obtained by selecting on L the view on the object types *users* and *files* and all event types. Give an accepting OCPN N_2 without duplicate labels such that, applying flattening-based conformance checking,
- the average alignment-based fitness of L_2 against N_2 is at least 90%, and
 - the average alignment-based precision of L_2 against N_2 is at least 50%.

Based on N_2 , use the ProM plugins **Replay a Log on Petri Net for Conformance Analysis** to compute optimal alignments and fitness scores on the subnets. Use these alignments with the plugin **Check Precision using Escaping Edges** to compute precision scores. Along with your report, submit the filtered event log L_2 , a figure of N_2 and the artifacts created in ProM. Explain your approach for finding N_2 (*approx. 2–3 sentences*).

*Hint: If you model N_2 by hand, you can use the plugin **Create Petri net (Text-language based)** to model a subnet, save the net with “Save DPN to ProM Workspace” and convert the resulting file into an accepting Petri net using the plugin **Create Accepting Petri net**.*

Total for Question 5: 12

¹<https://ocpq.aarkue.eu/docs/query-guide>

²As shown in the lecture, selecting a view means filtering out object and event types that are not selected.

References

- [1] Elvio Gilberto Amparore et al. “30 years of GreatSPN”. In: *Principles of Performance and Reliability Modeling and Evaluation*. 2016, pp. 227–254.